

Day 19 — Wed 23 Sep

Countdown: 3 days to General English & GK

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

● General — mock review + current affairs

Source: *General/03_GK_CAPSULE.md*

9.1 How current affairs is actually tested

In a state GK paper, current affairs is a small block of **objective recall** — "who won X", "which summit was held where", "which scheme was launched" — plus the odd short note. You do **not** need deep analysis. You need a curated list of the last ~12 months' headline facts, especially anything touching **Nagaland, the North-East, and India's big national events**.

● Core — normalisation

Source: *_Shared/CORE_06_DBMS.md*

10. Normalization

This is the highest-scoring theory-plus-numerical topic in the DBMS paper. Learn to work a decomposition by hand at every normal form.

Concept

Normalization is the process of organising attributes into relations to **reduce redundancy and eliminate update anomalies**, by successively applying stricter rules ("normal forms"). The enemy it fights is the **anomaly**:

- **Insertion anomaly** — you cannot add a fact without inventing unrelated data (can't record a new course with no student enrolled).
- **Update anomaly** — one fact stored many times; you must update every copy or risk inconsistency (change an instructor's phone in 40 rows).
- **Deletion anomaly** — removing one fact accidentally destroys another (deleting the last student on a course loses the course itself).

All three come from **redundancy**, and redundancy comes from **functional dependencies** that do not centre on a key. Fix the dependencies and the anomalies vanish.

Functional dependency (FD)

An FD $X \rightarrow Y$ ("X determines Y") means: whenever two tuples agree on X, they must agree on Y. X is the **determinant**. Example: `roll_no → name` (a roll number determines exactly one name).

Types:

- **Trivial FD:** $Y \subseteq X$ (e.g. `{roll_no, name} → name`) — always holds, uninteresting.
- **Non-trivial FD:** $Y \not\subseteq X$ — the useful kind.
- **Full FD:** Y depends on **the whole** of a composite X, not part of it.
- **Partial FD:** Y depends on **part** of a composite key — the 2NF villain.

- **Transitive FD**: $X \rightarrow Y$ and $Y \rightarrow Z$ give $X \rightarrow Z$ where Y is non-key — the 3NF villain.

Armstrong's axioms

The **sound and complete** inference rules for deriving all FDs implied by a set. The three primary axioms:

Axiom	Rule
Reflexivity	If $Y \subseteq X$ then $X \rightarrow Y$ (trivial dependencies)
Augmentation	If $X \rightarrow Y$ then $XZ \rightarrow YZ$ for any Z
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$

Derived (secondary) rules — handy shortcuts:

Rule	Statement
Union	If $X \rightarrow Y$ and $X \rightarrow Z$ then $X \rightarrow YZ$
Decomposition	If $X \rightarrow YZ$ then $X \rightarrow Y$ and $X \rightarrow Z$
Pseudo-transitivity	If $X \rightarrow Y$ and $WY \rightarrow Z$ then $WX \rightarrow Z$

Sound = derives only true FDs; **complete** = derives *all* true FDs. That phrase earns a mark.

Attribute closure (X^+)

X^+ is the set of **all attributes functionally determined by X** . Algorithm: start with $X^+ = X$; repeatedly, for every FD $A \rightarrow B$ where $A \subseteq X^+$, add B to X^+ ; stop when nothing new is added. Two uses:

1. **Is X a super key?** X is a super key **iff** $X^+ = \text{all attributes}$.
2. **Does $X \rightarrow Y$ hold?** Yes iff $Y \subseteq X^+$.

Worked closure. $R(A,B,C,D,E)$, FDs: $A \rightarrow B$, $B \rightarrow C$, $CD \rightarrow E$.

- $\{A\}^+$: start $\{A\}$; $A \rightarrow B$ adds $B \rightarrow \{A,B\}$; $B \rightarrow C$ adds $C \rightarrow \{A,B,C\}$; $CD \rightarrow E$ needs D (absent) $\rightarrow$ stop.

$\{A\}^+ = \{A,B,C\}$. Not a super key (D, E missing).

- $\{A,D\}^+$: $\{A,D\} \rightarrow +B \rightarrow +C$ (now $\{A,B,C,D\}$) $\rightarrow CD \rightarrow E$ adds $E \rightarrow \{A,B,C,D,E\} = \text{all}$. So **AD is a super key**, and since neither A nor D alone gives everything, **AD is a candidate key**.

Finding candidate keys — the reliable method

1. Find attributes that appear **only on the left** of FDs (or in no FD): they **must** be in every candidate key.
2. Attributes appearing **only on the right** are **never** in a candidate key.
3. Take the closure of the "must-be-in" set. If it is all attributes, that is the key. Otherwise add other attributes one at a time and test closure, keeping the sets **minimal**.

Worked key-finding. $R(A,B,C,D)$, FDs: $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$.

A appears only on the left $\rightarrow A$ is essential. $\{A\}^+ = \{A,B,C,D\} = \text{all}$. **A is the only candidate key**. B, C, D are prime nowhere else, so no other candidate key exists.

The normal forms — each with a worked decomposition

First Normal Form (1NF)

Rule: every cell holds a **single atomic value** — **no repeating groups, no multivalued attributes, no nested tables**.

Unnormalised (0NF):

```
STUDENT(roll_no, name, phones)
(1, Ravi, "9000,9001")      ← two phones in one cell
```

1NF — atomise (one row per value):

```
STUDENT(roll_no, name, phone)
(1, Ravi, 9000)
(1, Ravi, 9001)
```

The table is now 1NF but has introduced redundancy (**name** repeats), which the higher forms fix.

Second Normal Form (2NF)

Rule: 1NF and no partial dependency — every non-prime attribute is **fully** functionally dependent on **the whole** of every candidate key. (Only relevant when the key is **composite**; a single-attribute key is automatically in 2NF.)

Violating example — a store's line-item table:

```
R(order_id, product_id, qty, product_name, product_price)
Key = (order_id, product_id)
FDs: (order_id, product_id) → qty
      product_id → product_name, product_price  ← PARTIAL (depends on part of the key)
```

product_name depends only on **product_id**, not the whole key. **Decompose:**

```
ORDER_ITEM(order_id, product_id, qty)      -- full dependency only
PRODUCT(product_id, product_name, product_price)
```

Third Normal Form (3NF)

Rule: 2NF and no transitive dependency of a non-prime attribute on a candidate key. Equivalently, for **every** non-trivial FD $X \rightarrow Y$, **either X is a super key, or Y is a prime attribute** (part of some candidate key).

Violating example:

```
STUDENT(roll_no, name, dept_id, dept_head)
Key = roll_no
FDs: roll_no → dept_id ,   dept_id → dept_head
      ⇒ roll_no → dept_head  TRANSITIVELY (dept_id is non-key)
```

Decompose to remove the transitive dependency:

```
STUDENT(roll_no, name, dept_id)
DEPT(dept_id, dept_head)
```

Now **dept_head** is stored **once per department**, not once per student — the update anomaly is gone.

Boyce-Codd Normal Form (BCNF)

Rule: for **every** non-trivial FD $X \rightarrow Y$, **X must be a super key**. BCNF is stricter than 3NF; the only difference is that 3NF forgives an FD whose right side is a *prime* attribute, BCNF does not. So **3NF $\subset$ BCNF may fail only when there are overlapping candidate keys**.

Classic violating example:

```
R(student, subject, teacher)
Rules: each teacher teaches exactly ONE subject    → teacher → subject
      each (student, subject) has one teacher      → (student, subject) → teacher
Candidate keys: (student, subject) and (student, teacher)
```

`teacher` $\rightarrow$ `subject` has a determinant (`teacher`) that is **not a super key** $\rightarrow$ violates BCNF (but the table *is* in 3NF, because `subject` is a prime attribute). **Decompose:**

```
R1(teacher, subject)      -- teacher  $\rightarrow$  subject
R2(student, teacher)     -- who is taught by whom
```

The 3NF/BCNF trade-off to state: decomposition into BCNF is always **lossless**, but may **not preserve all dependencies**; 3NF can always be achieved **losslessly and dependency-preserving**. That is precisely why 3NF is the practical target and BCNF the ideal.

Fourth Normal Form (4NF)

Rule: BCNF and no non-trivial multivalued dependency (MVD) $X \twoheadrightarrow Y$ unless X is a super key.

An **MVD** $X \twoheadrightarrow Y$ means X determines a *set* of Y values **independently** of the other attributes. It appears when one table crams **two independent multivalued facts** together.

Violating example — a person's skills and languages are independent:

```
R(emp, skill, language)
emp  $\twoheadrightarrow$  skill and emp  $\twoheadrightarrow$  language (independent)
(1, Java, English)
(1, Java, Hindi)
(1, Python, English)
(1, Python, Hindi)  ← every skill × every language must be listed: redundancy
```

Decompose into two tables, one per independent fact:

```
EMP_SKILL(emp, skill)
EMP_LANG(emp, language)
```

(Beyond 4NF: **5NF/PJNF** handles join dependencies; rarely examined — mention it exists.)

Summary ladder

Form	Removes	One-line rule
1NF	Repeating groups / multivalued cells	All values atomic
2NF	Partial dependencies	No non-prime attribute depends on part of a composite key
3NF	Transitive dependencies	Every determinant is a super key , <i>or</i> the dependent is prime
BCNF	Remaining anomalies from overlapping keys	Every determinant is a super key
4NF	Multivalued dependencies	No non-trivial MVD unless the determinant is a super key

Lossless join and dependency preservation

When you split R into R_1 and R_2 , two properties matter:

Lossless (non-additive) join: re-joining R_1 and R_2 must give back **exactly** R — no spurious

extra rows. **Test:** the decomposition of R into R_1 , R_2 is lossless **iff** the common

attributes ($R_1 \cap R_2$) form a **super key of R_1 or of R_2** — i.e. $(R_1 \cap R_2) \rightarrow R_1$ or

$(R_1 \cap R_2) \rightarrow R_2$. A lossy decomposition invents rows that were never there and is unacceptable.

Worked test. $R(A,B,C)$, FDs $A \rightarrow B$, $B \rightarrow C$. Split into $R_1(A,B)$ and $R_2(B,C)$. Common attribute = $\{B\}$.

Is B a key of R_2 ? $B \rightarrow C$ so $\{B\}^+ \supseteq \{B,C\}$ = all of $R_2 \rightarrow$ **yes**. Decomposition is **lossless**. ✓

Dependency preservation: every original FD should be enforceable by checking **one table**

alone, without a join. If $(F_1 \cup F_2)^+ = F^+$, dependencies are preserved. In the example above,

$A \rightarrow B$ lives in R_1 and $B \rightarrow C$ lives in R_2 — **both preserved**. ✓

Likely exam questions

- **[5]** Define functional dependency. Differentiate full, partial and transitive dependencies.
- **[5]** State Armstrong's axioms. What does it mean that they are sound and complete?
- **[5]** Compute the closure $\{A\}^+$ for the given FD set and state whether A is a candidate key.
- **[5]** Differentiate between 3NF and BCNF with an example.
- **[5]** What is a lossless-join decomposition? State the test.
- **[10]** Explain 1NF, 2NF and 3NF with a single running example, decomposing at each step.
- **[15]** What is normalization? Explain 1NF to BCNF with worked decompositions, and discuss lossless join and dependency preservation, stating why 3NF is usually preferred to BCNF.

MCQ traps

Trap	Correct answer
"1NF eliminates?"	Repeating groups / multivalued attributes.
"2NF removes?"	Partial dependency; needs a composite key to be at risk.
"3NF removes?"	Transitive dependency.
"BCNF requires every determinant to be a?"	Super key.
"4NF deals with?"	Multivalued dependencies (MVD).
"Which is stronger, 3NF or BCNF?"	BCNF.
"3NF vs BCNF — which guarantees dependency preservation?"	3NF.
"Armstrong's axioms are?"	Sound and complete.
" X^+ = all attributes means X is a?"	Super key.
"A decomposition is lossless if the common attributes are a?"	Super key of one of the tables.